

Quaternions in Practice

Converting, Validating, and Understanding

Ben Kenwright

January 2013

Abstract In this paper, we present a practical overview of the advantages and disadvantages of quaternions for creating computationally fast, straightforward, and algorithmically robust solutions. While quaternions have proven themselves as providing an unambiguous, un-cumbersome, and computationally efficient tool for representing rotations, we attempt to address a number of workable considerations (e.g., converting between different forms, numerical errors, and common pitfalls). Whereby, this paper focuses on providing a clear-cut step-by-step explanation of quaternions with real-world examples and practical considerations.

Keywords Quaternions · Simulation · Converting · Euler Angles · Axis-Angle · Quaternion Exponential

1 Introduction

Overview: Quaternions are a memory efficient, straightforward, and computationally robust way of representing orientations. The algebraic properties of quaternions allows them to perform important operations (e.g., interpolation and inversion) that would be difficult and imprecise by other means (e.g., compared with axis-angle and matrix methods). Quaternions have been around for over a century and have become the de-facto method of representing, comparing, and storing orientations. Possessing a solid understanding of quaternions, which include their practical workings, limitations, and pitfalls is fundamental.

Motivation (Interest and Importance): Representing an orientation efficiently and without ambiguity is important in many disciplines (e.g., robotics, computer-animation, and

bio-mechanics). While quaternion libraries are freely available which provide a broad range of optimized API, the user can use the quaternion tool with a blind eyes to the mechanical workings within. However, when problems occur or customization is needed (e.g., how to identify if the quaternion is valid, porting the quaternion solution to another platform), the user is left struggling and guessing.

Challenges: Ideally, we want to represent orientations in an unambiguous and memory efficient way. Furthermore, performing a number of operations, such as interpolation, inversion, and concatenation of orientations should be possible without too much difficulty. While quaternions solve the majority of the problems there is a voodoo surrounding them. For example, why do people fear quaternions? While the name *quaternion* is a daunting name to begin with some students struggled with complex number at school, so the thought of understanding a 4D space with multiple variable complex numbers can leave the majority of students staying clear of the subject. Nevertheless, while quaternions solve a number of problems and provide us with a powerful tool, the human brain just cannot visualize 4D and it can make visualizing and comprehending quaternions difficult.

Quaternion Approach: While quaternions may, initially, appear to be the holy-grail solution for fast, compact, unambiguous orientations, it does, however, have some disadvantages and heartfelt traps for those unaccustomed to working with them. Whereby, this paper aims to help the reader appreciate, understanding, and identifying the many advantages while demonstrating and explaining to the reader how to comprehend and circumventing the few disadvantages.

Key Components and Results: This paper explores a number of practical real-world implementation situations involv-

ing quaternions. For example, converting to and from alternative formats, such as Euler angles, matrices, and axis-angles. We present experimental results and simplified step-by-step examples to help solidify and prove certain theoretical aspects.

Contribution: The contributions of this paper is a practical non-nonsense explanation and examination of quaternions in a real-world context. We present numerous step-by-step explanations of the mathematical intricacies of quaternions and their conversion to and from other rotational representations (e.g., matrices, axis-angle, Euler angles).

2 Related Work

The subject of quaternions is not a new topic and numerous interesting and comprehensive texts on the subject are available[xxxx]. However, realistic pedestrians in crowd situations is an interesting subject which has been studied across numerous disciplines (e.g., movie industry, computer games, safety and evacuation). Whereby, we briefly review some of the most recent and relevant research in the field that has contributed to making characters and crowds more *interactive and responsive*.

Beyond quaternions - to dual-quaternions!

3 Fundamentals

We review the quaternions fundamental representation and its algorithmic operations.

Quaternion Form:

$$\begin{aligned} q &= [w + xi + yj + zk] & w, x, y, z &\in \mathbb{R} \\ &= [w, \mathbf{v}] & \mathbf{v} &= (x, y, z), \mathbf{v} \in \mathbb{R}^3 \end{aligned} \quad (1)$$

Complex Algebraic Rules:

$$\begin{aligned} i^2 &= j^2 = k^2 = -1 \\ ij &= k & ik &= i & ki &= j \\ ji &= -k & kj &= -i & ik &= j \end{aligned} \quad (2)$$

Hint, they follow the same rules as the cross-product, i.e.:

$$\begin{aligned} x \times y &= z & y \times z &= x & z \times x &= y \\ y \times x &= -z & z \times y &= -x & x \times z &= -y \end{aligned} \quad (3)$$

Addition and Subtraction:

$$\begin{aligned} q_0 &= [w_0, \mathbf{v}_0] \\ q_1 &= [w_1, \mathbf{v}_1] \\ q_0 + q_1 &= [w_0 + w_1, \mathbf{v}_0 + \mathbf{v}_1] \\ q_0 - q_1 &= [w_0 - w_1, \mathbf{v}_0 - \mathbf{v}_1] \end{aligned} \quad (4)$$

Product of Two Quaternions:

$$\begin{aligned} q_0 q_1 &= [w_0, \mathbf{v}_0][w_1, \mathbf{v}_1] \\ &= [w_0 w_1 - \mathbf{v}_0 \cdot \mathbf{v}_1, \\ &\quad w_0 \mathbf{v}_1 + w_1 \mathbf{v}_0 + \mathbf{v}_0 \times \mathbf{v}_1] \end{aligned} \quad (5)$$

Real Quaternions:

$$q = [w, 0] \quad (6)$$

Product of 'two' real quaternions:

$$[\mathbf{w}_0, 0][\mathbf{w}_1, 0] = [\mathbf{w}_0 \mathbf{w}_1, 0] \quad (7)$$

Multiplying by a Scalar:

$$\begin{aligned} q &= [w, \mathbf{v}] \\ \beta q &= \beta[w, \mathbf{v}] \\ &= [\beta w, \beta \mathbf{v}] \end{aligned} \quad (8)$$

which can be conformed by multiplying by a 'real' quaternion with the scalar component as the value (i.e., $[a, 0][w, \mathbf{v}] = [aw, a\mathbf{v}]$).

Pure Quaternions:

$$q = [0, \mathbf{v}] \quad (9)$$

Product of 'two' pure quaternions:

$$[0, \mathbf{v}_0][0, \mathbf{v}_1] = [-\mathbf{v}_0 \cdot \mathbf{v}_1, \mathbf{v}_0 \times \mathbf{v}_1] \quad (10)$$

Sum of Pure and Real Quaternions:

$$\begin{aligned} q &= [w, \mathbf{v}] \\ &= [w, 0] + [0, \mathbf{v}] \end{aligned} \quad (11)$$

Conjugate:

$$\begin{aligned} q &= [w, \mathbf{v}] \\ q^* &= [w, -\mathbf{v}] \quad * \text{ indicates the conjugate} \end{aligned} \quad (12)$$

whereby, a quaternion multiplied with its conjugate is:

$$\begin{aligned} qq^* &= [w, \mathbf{v}][w, -\mathbf{v}] \\ &= [w^2 - \mathbf{v} \cdot -\mathbf{v}, -s\mathbf{v} + s\mathbf{v} + \mathbf{v} \times -\mathbf{v}] \\ &= [w^2 + \mathbf{v} \cdot \mathbf{v}, 0] \\ &= [w^2 + \mathbf{v}^2, 0] \end{aligned} \quad (13)$$

Magnitude (or Norm):

$$\begin{aligned} q &= [w, \mathbf{v}] \\ ||q|| &= \sqrt{w^2 + \mathbf{v}^2} \quad \text{Magnitude (or Norm)} \\ qq^* &= ||q||^2 \end{aligned} \quad (14)$$

Normalizing:

$$\begin{aligned} q' &= \frac{q}{||q||} \\ &= \frac{[w, \mathbf{v}]}{\sqrt{w^2 + \mathbf{v}^2}} \end{aligned} \quad (15)$$

Inverse:

$$q^{-1} = \frac{q^*}{||q||^2} \quad (16)$$

$$qq^{-1} = [1, 0] = 1$$

For a unit-quaternion whose norm is 1, the conjugate is the same as the inverse, i.e., $q^{-1} = q^*$.

Dot-Product: The quaternion dot-product is analogous to the vector dot-product. The two quaternion have their corresponding scalar components multiplied together and the summed result is the answer:

$$q_0 \cdot q_1 = [w_0 w_1 + x_0 x_1 + y_0 y_1 + z_0 z_1]$$

$$\cos \theta = \frac{q_0 \cdot q_1}{||q_0|| ||q_1||} \quad (17)$$

where θ is the angle between the two quaternions and for a unit quaternion $\cos \theta = q_0 \cdot q_1$.

4 Practical Algorithms**5 Gimbal Lock**

Euler angles possess singularity problems.

6 Quaternion from two points (i.e. a direction to quaternion)

Taken to points in 3D space we can calculate a direction (or aim) vector that we can use to formulate an orientation quaternion. However, a direction vector does not possess any twist information. Furthermore, the quaternion orientation from the direction vector is in world space but can be re-orientated relative to a local space through quaternion multiplication (e.g., to joint space). The orientation aims the object along the z-axis based on a direction vector (d) shown in Equation 18.

$$\mathbf{q} = (q_w, q_x, q_y, q_z) \quad (18)$$

$$= (1 - d_z, d_x, -d_y, 0)$$

where d is a direction vector. However, there is a singularity when the direction vector (d) is aligned with the z-axis that must include a special case check (shown in Listing 1).

We can visualize the resulting quaternion orientation information by drawing the generated primary rotation axis on the surface of a sphere, see Figure 1

```
Quaternion QuaternionFromDirection( Vector3 p0, ←
    Vector3 p1 )
{
    Vector3 v = Vector3.Normalize( p1 - p0 );
    if ( v.Z > 0.999f )
    { // xyzw
        return new Quaternion( 0, 1, 0, 0 );
    }
    Quaternion qr =
    new Quaternion( v.Y, -v.X, 0, 1 - v.Z );
    qr = Quaternion.Normalize( qr );
}
```

```
Debug.c.Valid( qr );
return qr;
}
```

Listing 1 Orientation from Directoin - Function source code for calculating an orientation quaternion from a direction composed of two points.

7 Quaternion to Euler

While quaternions are mathematically great, they can be difficult to visualize and understand with the minds eye. If needed, however, we can quickly convert a quaternion to its Euler angle components to aid in debugging or visualizing the orientations (as shown below in Listing 3).

```
Vector3 QuatToEuler( Quaternion q )
{
    double sqw = q.W * q.W;
    double sqx = q.X * q.X;
    double sqy = q.Y * q.Y;
    double sqz = q.Z * q.Z;

    double tmp =
    Clamp( -2.0 * ( q.X * q.Z - q.Y * q.W ), -1, 1 );

    double rotxrad =
    Math.Atan2( 2.0 * ( q.Y * q.Z + q.X * q.W ), ( -sqx ←
    - sqy + sqz + sqw ) );

    double rotyrad =
    Math.Asin( tmp );

    double rotzrad =
    Math.Atan2( 2.0 * ( q.X * q.Y + q.Z * q.W ), ( sqx ←
    - sqy - sqz + sqw ) );

    Vector3 euler = new Vector3( (float)rotxrad,
    (float)rotyrad,
    (float)rotzrad );

    Debug.c.Valid( euler );
    return euler;
}
```

Listing 2 Quaternion to Euler Angles - Function source code for calculating XYZ Euler angles from a unit-quaternion orientation.

7.1 Verification

When converting to or from the quaternion format always do the reverse operation and check that its what you began with (i.e., in debug).

8 Angular Limits**9 Component Decomposition****9.1 Axis-Angle**

Quaternion from Axis-Angle: Given a axis of rotation and an angle, we can construct a quaternion:

$$q = \left[\cos \left(\frac{\theta}{2} \right), \mathbf{n} \sin \left(\frac{\theta}{2} \right) \right] \quad (19)$$

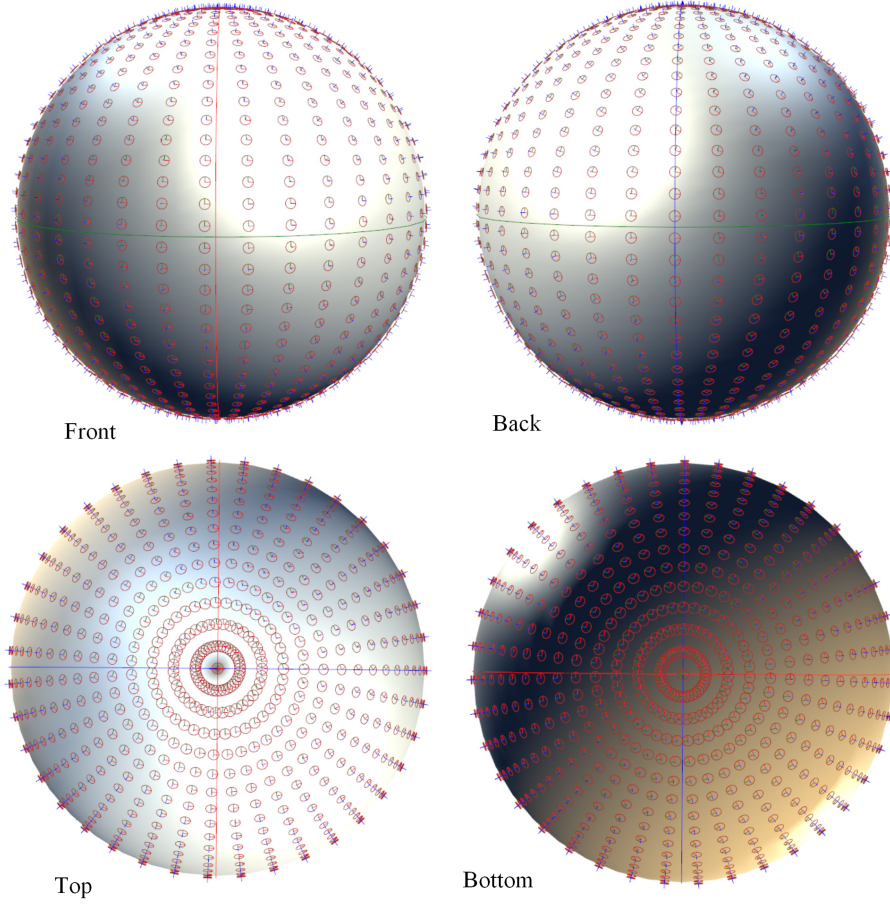


Fig. 1 Orientation Visualization - Orientation information drawn on a sphere's surface using Equation 18.

Quaternion to Axis-Angle:

$$\begin{aligned}\theta &= 2 \cos^{-1}(q_w) \\ n_x &= \frac{q_x}{\sin(\theta)} \\ n_y &= \frac{q_y}{\sin(\theta)} \\ n_z &= \frac{q_z}{\sin(\theta)}\end{aligned}\quad (20)$$

Warning - make sure the value for acos is between -1 to 1, since numerical errors, such as -1.00001 will cause acos to trigger a nan (not a number).

9.2 Matrices

Quaternion to Matrix:

$$\begin{bmatrix} 1 - 2q_y^2 - 2q_z^2 & 2q_xq_y - 2q_zq_w & 2q_xq_z + 2q_yq_w \\ 2q_xq_y + 2q_zq_w & 1 - 2q_x^2 - 2q_z^2 & 2q_yq_z - 2q_xq_w \\ 2q_xq_z - 2q_yq_w & 2q_yq_z + 2q_xq_w & 1 - 2q_x^2 - 2q_y^2 \end{bmatrix}\quad (21)$$

Quaternion from Matrix: A rotation may be converted back to a quaternion. First, calculate the trace of the matrix T from the equation:

$$\begin{bmatrix} m0 & m1 & m2 \\ m3 & m4 & m5 \\ m6 & m7 & m8 \end{bmatrix}\quad (22)$$

$$\begin{aligned}\text{Trace } T &= 4 - 4x^2 - 4y^2 - 4z^2 \\ &= 1 + m0 + m4 + m8\end{aligned}\quad (23)$$

If the trace of the matrix is greater than zero, then perform an "instant" calculation. Important note wrt. rounding errors: Test if ($T \geq 0.00000001$) to avoid large distortions!

$$\begin{aligned}S &= \sqrt{T} * 2; \\ W &= 0.25 * S; \\ X &= (m7 - m5)/S; \\ Y &= (m2 - m6)/S; \\ Z &= (m3 - m1)/S;\end{aligned}\quad (24)$$

If the trace of the matrix is equal to zero then identify which major diagonal element has the greatest value. Depending on this, calculate the following:

```

if ( m0 > m4 && m0 > m8 ) { // Column 0:
    S = sqrt( 1.0 + m0 - m4 - m8 ) * 2;
    X = 0.25 * S;
    Y = (m4 + m1) / S;
    Z = (m2 + m6) / S;
    W = (mat[9] - m5) / S;
} else if ( m4 > m8 ) { // Column 1:
    S = sqrt( 1.0 + m4 - m0 - m8 ) * 2;
    X = (m3 + m1) / S;
    Y = 0.25 * S;
    Z = (m7 + m5) / S;
    W = (m2 - m6) / S;
} else { // Column 2:
    S = sqrt( 1.0 + m8 - m0 - m4 ) * 2;
    X = (m2 + m6) / S;
    Y = (m7 + m5) / S;
    Z = 0.25 * S;
    W = (m3 - m1) / S;
}

```

Listing 3 Quaternion to Euler Angles - Function source code for calculating XYZ Euler angles from a unit-quaternion orientation.

9.3 Logs and Exp

A truly beautiful and interesting feature of quaternions is what happens when we take the log of a *unit-quaternion*. The logarithm (i.e., log) of a unit-quaternion is another quaternion; however, the quaternion vector component is the axis-angle combination and the scalar component is zero [Ebe05].

$$\begin{aligned}
 \log(q) &= \log([q_w, q_x, q_y, q_z]) \\
 &= \log([\cos\theta, \hat{u} \sin\theta]) \\
 &= [0, \hat{u}\theta]
 \end{aligned} \tag{25}$$

Similarly, the exponential (i.e., exp) of a quaternion is the mirror opposite of the logarithm.

$$\begin{aligned}
 \exp([0, \hat{u} \theta]) &= q \\
 &= [q_w, q_x, q_y, q_z] \\
 &= [\cos\theta, \hat{u} \sin\theta]
 \end{aligned} \tag{26}$$

$$\begin{aligned}
 \exp(\log(q)) &= q \\
 \log(\exp([0, \hat{u} \theta])) &= [0, \hat{u} \theta]
 \end{aligned} \tag{27}$$

This remarkably simple and useful feature is proven by means the Taylor power series for the exponential, and allows us to take a quaternion to a scalar *power* (i.e., it makes operations such as interpolation very straightforward and algorithmically elegant).

10 Interpolation

Interpolation is the process of finding a value between two known points. These points maybe be a line or they may be the surface of a sphere. The interpolation value is typically between 0 and 1, where 0 is the start point, 1 is the end point, and 0.5 would be half way inbetween.

10.1 [L]inear Int[erp]olation (LERP)

$$q' = q_0 + (q_1 - q_0)t \quad (t \in \mathbb{R}) \tag{28}$$

10.2 [S]pherical [L]inear Int[erp]olation (SLERP)

$$Slerp(t; q_0, q_1) = \frac{q_0 \sin((1-t)\theta) + q_1 \sin(t\theta)}{\sin\theta}$$

where $0 \leq t \leq 1$

(29)

10.2.1 Pitfalls - Not always the shortest path

Naively spherically interpolating between two quaternions using Equation 29 will not necessarily take you along the *shortest* path. Since, we might not always want it to take the shortest path, we must identify the shortest and longest. First, a quaternion can be negated and maintain the same unique orientation:

$$\begin{aligned}
 q &= -q \\
 [w, x, y, z] &= [-w, -x, -y, -z]
 \end{aligned} \tag{30}$$

Negating a quaternion enables makes a quaternion take either the longest or shortest path. We can determine which route by means of the quaternion dot product.

$$\begin{aligned}
 q_0 \cdot q_1 &= q_0.w * q_1.w + \\
 &\quad q_0.x * q_1.x + \\
 &\quad q_0.y * q_1.y + \\
 &\quad q_0.z * q_1.z
 \end{aligned} \tag{31}$$

where the dot product of two quaternions gives us the cosine of the angle between the two rotations. Hence, for the 'shortest' path, we choose dot product result that is greater or equal to zero. **Remember between $q_0 \cdot q_1$ and $q_0 \cdot -q_1$ the one that is greater or equal to zero (i.e. ≥ 0) is the shortest route between the two quaternions.**

```

Quaternion ShortSLERP( Quaternion q0, Quaternion q1, ←
    float t )
{
    // Assume unit-quaternions of length one
    float dot = Dot(q1, q2);
    float omega = ArcCos( dot );
    float sign = dot > 0 ? 1 : -1;
    Quaternion res
    = (q0 * Sin((1-t)*omega) + sign*q1*Sin(t*omega)) / Sin(←
        omega );
    return res;
}

```

Listing 4 Quaternion Shortest Route SLERP - Function to calculating and always return the shortest SLERP between two unit-quaternions.

10.2.2 Divide-by-Zero

A problem with SLERP that people ignore or forget is that for extremely small angles or an angle of zero you can get numerical problems (e.g., a divide by zero error). Did you notice the “divide by $\sin\theta$ ”? Hence, for small angles less than some specific amount you would use a normalized LERP method (i.e., just interpolate each of the quaternion components and re-normalize the results so it is a unit-quaternion).

11 Calculus

Integration and differentiation might seem daunting and complex but, in fact, both are relatively straightforward and can be deduced from first principles. For example, we show how to derive the well-known differential form for a quaternion, i.e., $\dot{q} = 0.5 \omega q$, which is often used to represent angular velocity (Section 11.1).

11.1 Quaternion Differentiation

A sweet and short explanation for differentiation of a quaternion. First the definition of what we mean by ‘differentiate’ (i.e., from first principles):

$$\dot{q} = \frac{dq}{dt} = \lim_{h \rightarrow 0} \frac{q(t+h) - q(t)}{h} \quad (32)$$

which is the same for any real or complex function.

We are only concerned with unit-quaternions. The unit-quaternion represents the orientation. When we differentiate the unit-quaternion we are describing how the orientation changes with time (i.e., the angular velocity). Multiplying two unit-quaternions results in a unit-quaternion (e.g., $q = q_0 q_1$). Hence, we can describe the orientation now and at the next moment in time as (assuming a **constant angular velocity** ω):

$$\begin{aligned} q(0) &= q_0 \\ q(1) &= q_\omega q_0 \end{aligned} \quad (33)$$

where q_ω rotate in one time-step.

Recall, we have a *constant* angular velocity, so we can say:

$$\begin{aligned} q(2) &= q_\omega q_\omega q_0 \\ q(3) &= q_\omega q_\omega q_\omega q_0 \\ &\dots \\ q(k) &= (q_\omega)^k q_0 \end{aligned} \quad (34)$$

Remember, we are dealing with ‘unit-quaternions’, so q_ω in *polar* form is:

$$q_\omega = \cos\left(\frac{\theta}{2}\right) + \sin\left(\frac{\theta}{2}\right) (\mathbf{n}_x i + \mathbf{n}_y j + \mathbf{n}_z k) \quad (35)$$

where ω is the angle, and $\mathbf{n}$ is the unit-vector axis of rotation. Important information is we are dealing with a polar form of the equation - essentially, a quaternion version of Euler’s formula. A unit complex number z can be written as:

$$z = \exp(it) = \cos(t) + i \sin(t) \quad (36)$$

similarly for a unit-quaternion:

$$q_\omega = \exp\left(\frac{\theta}{2} \mathbf{n}\right) \quad (37)$$

hence, we deduce:

$$(q_\omega)^t = \exp\left(t \frac{\theta}{2} \mathbf{n}\right) \quad (38)$$

we use the invaluable and popular differential *chain-rule* and *product-rule* formula:

$$\begin{aligned} \dot{q}(t) &= \frac{d}{dt} (q_\omega^t q_0) \\ &= \frac{\theta}{2} \mathbf{n} \exp\left(t \frac{\theta}{2} \mathbf{n}\right) q_0 \\ &= \frac{\theta}{2} \mathbf{n} q(t) \end{aligned} \quad (39)$$

where $\theta \mathbf{n}$ is the angular velocity ω and yields the neat and popular equation:

$$\begin{aligned} \dot{q}(t) &= \frac{dq}{dt} \\ &= \frac{1}{2} (\theta \mathbf{n}) q \\ &= \frac{1}{2} \omega q \end{aligned} \quad (40)$$

12 Experimental Results

Theoretically, quaternions are able to perform simple rotational operations faster than pure rotation matrices. However, in practice, quaternions are regularly converted to matrices since transforms usually require translation and scaling in addition to rotation and for visual rendering the graphical processing unit (GPU) is highly optimized for matrix operations.

Performance: Nevertheless, in reality quaternions are fast. We can perform operations such as concatenation, inversion, interpolation, and re-normalization extremely quickly.

Robustness: Quaternions are not infallible and can suffer from numerical drifting. Furthermore, they only represent orientations and cannot represent full rigid transforms that include translation. (see Section 14 on dual-quaternions for translation and rotation beyond quaternions for more details).

Scalability and Sensitivity: Numerical errors can drift in over time and so we must constantly check if our quaternion is a unit-quaternion (i.e., a unit-length). In practice, floats are used to represent the four scalar values (32 bit accuracy).

13 Limitations and Assumptions

Quaternions can be difficult for a beginner to understand, however, they provide numerous advantages over Euler angles and matrices for representing rotations.

- ✓ Rotation concatenation (i.e., it's faster than matrix rotation concatenation)
- ✓ Quaternion to Matrix (slightly faster than Euler angles to matrix conversion)
- ✓ Inverse of a unit-quaternion is the conjugate (extremely fast and simple)
- ✓ Quaternion only requires four scalar numbers, while a rotation matrix requires nine
- ✓ Interpolation is fast, smooth, and constant using quaternion SLERP
- ✓ Correcting for numerical errors, i.e., by re-normalizing the quaternion is less error prone and computationally simpler compared to the similar process of orthonormalizing a rotation matrix
- ✗ Quaternions can drift and become invalid; hence, we need to be aware of this and re-normalize them
- ✗ The greatest drawback of quaternions is people find them difficult to understand.

14 Dual-Quaternions

While quaternions have proven themselves as an unambiguous, computationally efficient, and straightforward tool for representing rotations, they provide no translational information. However, one approach has combined quaternions with dual-number theory to create dual-quaternions. Dual-quaternions exploit the elegant mathematical properties of quaternions and combine them with dual-number theory to create a unified mathematical tool for representing both orientation and translation in a single variable; see Kenwright [Ken12] for further details.

15 Conclusions

There are numerous free and commercial quaternion libraries. It is also relatively straightforward to write your own basic class. However, writing a robust, computationally lightening fast implementation, takes time and experience. I would recommend to the reader to always look under the hood of other implementations to pick up tricks and ideas.

16 Acknowledgments

The author would like to thank the anonymous reviewers for taking time out of their busy schedules to provide helpful criticism and comments to make this a better paper.

References

- Ebe05. EBERLY D.: Quaternions algebra and calculus. *Geometric Tools, LLC* (Nov. 2005). [5](#)
- Ken12. KENWRIGHT B.: A beginners guide to dual-quaternions: What they are, how they work, and how to use them for 3d character hierarchies. *The 20th International Conference on Computer Graphics, Visualization and Computer Vision WSCG 2012 Communication Proceedings* (2012), 1–13. [7](#)